

GITHUB-NATIVE CODE QUALITY

SonarQube Replacement POC

Implemented Controls, Runtime Evidence, Screenshots, Mapping Status, and Retirement Decision

DECISION

Conditionally approve the GitHub-native pattern as a replacement for SonarQube for eligible GitHub-hosted repositories. Security scanning, SonarQube-only governance metrics remain explicit exceptions.

Repository

ms-pwc/github-code-quality

Demonstration pull request

#1 - intentionally blocked by security and coverage controls

Assessment date

18 August 2026

Classification: Internal proof of concept. Deliberately vulnerable code; never deploy.

Repository: <https://github.com/ms-pwc/github-code-quality>

Live POC pull request: <https://github.com/ms-pwc/github-code-quality/pull/1>

Table of contents

Table of contents	1
1. Document purpose and control.....	1
2. Executive decision.....	1
What is proven	1
What is not proven or not equivalent	1
3. Implemented GitHub-native architecture.....	1
4. Test design and source equivalence.....	1
5. Detailed mapping implementation matrix	1
6. GitHub Code Quality configuration proof	1
7. Security implementation and CodeQL proof.....	1
Planned security scenarios not observed as CodeQL alerts	1
8. Reliability and maintainability proof.....	1
Quality cases not observed	1
9. Coverage implementation and gate proof.....	1
10. Quality gate and merge-control proof.....	1
11. Complementary GitHub-native security controls	1
12. SonarQube benchmark result on the same source	1
13. Explicit NOT DONE and PARTIAL register	1
14. SonarQube retirement decision framework	1
Do not remove SonarQube when any condition below is true	1
Recommended production rollout	1
15. Demonstration script	1
16. Evidence and navigation index.....	1
17. Official product references.....	1

1. Document purpose and control

This report updates the supplied ‘SonarQube to GitHub Capability Mapping’ with the actual implementation and runtime evidence from this repository. Every capability is labelled DONE, PARTIAL, or NOT DONE. A documented capability is never presented as executed proof.

Field	Value
Reference mapping	SonarQube-to-GitHub-CodeQL-Code-Quality-Detailed-Mapping.docx
POC language	C# / .NET 9
Safe baseline	main branch; 6 tests; 100% covered application logic after bootstrap exclusion
GitHub POC branch	github-native-poc
SonarQube POC branch	sonarqube-poc
Evidence head	5326bdf8ab636b4f3a078c52a5552549b449f51e
Evidence date	18 August 2026
Credential handling	No token or password is included in this report or committed evidence

2. Executive decision

Recommended decision: YES, with stated conditions

For this C# repository and the demonstrated pre-merge workflow, SonarQube can be removed and replaced by the combined GitHub-native p secret/dependency controls. This is not an unconditional claim of one-to-one product parity.

What is proven

- CodeQL blocked the pull request with 11 security alerts: 2 critical, 4 high, and 5 medium.
- GitHub Code Quality posted 10 deterministic inline findings: 9 reliability and 1 maintainability.
- GitHub built-in coverage reported 3% on the POC branch versus a 100% default-branch baseline.
- An explicit 80% coverage workflow check failed as designed.
- The active ruleset requires PR review, CI, dependency review, CodeQL results, and blocks high-or-higher security alerts.
- Secret scanning, push protection, Dependabot security updates, dependency review, and dependency graph/SBOM are enabled.
- The pull request merge state is BLOCKED.

What is not proven or not equivalent

- No GitHub-native first-class duplication percentage or duplication gate was implemented.
- No aggregate cyclomatic/cognitive complexity metric or gate was implemented.
- No SonarQube-equivalent remediation-time technical-debt model was implemented.
- No single centralized, inherited per-language Quality Profile equivalent exists in this POC.
- The exact SonarQube Security Hotspot Reviewed/Safe workflow is not reproduced by GitHub alert triage.
- Organization/portfolio historical reporting was not implemented or validated.
- Custom CodeQL packs, AI findings, and Copilot Autofix were not enabled or validated.

Retirement guardrail

Do not remove SonarQube from a production repository if any NOT DONE item is a mandatory audit, contract, or engineering requirement. Ad

3. Implemented GitHub-native architecture

Layer	Implemented component
Security analysis	CodeQL advanced setup / security-extended
Quality analysis	GitHub Code Quality managed C# analysis
Coverage	Coverlet -> Cobertura -> actions/upload-code-coverage@v1
Coverage enforcement	Test-CoverageThreshold.ps1 required CI check
Merge enforcement	Repository ruleset 20972666
Secrets	Secret scanning + push protection
Dependencies	Dependabot + dependency review + SBOM
Ownership	CODEOWNERS and one approval

GitHub-native branch: <https://github.com/ms-pwc/github-code-quality/tree/github-native-poc>

CodeQL workflow: <https://github.com/ms-pwc/github-code-quality/blob/github-native-poc/.github/workflows/codeql.yml>

CI and coverage workflow: <https://github.com/ms-pwc/github-code-quality/blob/github-native-poc/.github/workflows/ci.yml>

Dependency review workflow: <https://github.com/ms-pwc/github-code-quality/blob/github-native-poc/.github/workflows/dependency-review.yml>

Dependabot configuration: <https://github.com/ms-pwc/github-code-quality/blob/github-native-poc/.github/dependabot.yml>

4. Test design and source equivalence

Both tool branches contain byte-identical defect-corpus files. Git blob identity proves that the compared source is the same; tool-specific workflow, script, and evidence commits are separate.

Shared file	Git blob SHA-1
QualityDemo.Api.csproj	c82754a50cbd18b690f0f4dcf7880aaffc1b0006
VulnerableController.cs	b8843e01b8df74a15c0565c8bddf9bc4d581ff44
SecurityHotspotExamples.cs	71edfef006f8720d4b51c5cee979d4c22ab55f7f
ReliabilityExamples.cs	f55e1978e796998e2b60d65b09d7c359beda4766
MaintainabilityExamples.cs	063c8a24549a433d6a366c97d1fa4e3cd0ee1089
DuplicatedBusinessRules.cs	b9fc563673bd3e5b380346a91bb758fa22fcc480

- The sample is intentionally non-deployable.
- Six tests execute only SafeCalculator; no vulnerable endpoint is invoked.
- Credential-like strings are synthetic POC placeholders.
- The GitHub PR remains open to preserve failing checks and annotations as evidence.

5. Detailed mapping implementation matrix

Status definitions: DONE = implemented and observed; PARTIAL = useful substitute exists but is not equivalent or not fully exercised; NOT DONE = not implemented or not proven in this repository.

Mapping capability	GitHub implementation	Status	POC proof
Vulnerabilities	CodeQL code scanning	DONE	11 alerts; sem
Security Hotspots	CodeQL review/dismissal	PARTIAL	ECB and cook
Bugs / reliability	GitHub Code Quality	DONE	9 inline reliabi
Code smells	Code Quality maintainability	PARTIAL	1 maintainabi
Maintainability	Findings and default-branch score	PARTIAL	PR comments
Coverage visibility	Built-in Cobertura upload	DONE	3% branch ver
Coverage gate	Required custom workflow check	DONE	80% thresholc
Quality gate outcome	Ruleset + required checks + CodeQL	DONE	mergeStateSt
Native Code Quality threshold	Require code quality results	NOT DONE	Managed anal
Duplication	No first-class equivalent implemented	NOT DONE	SonarQube: 2
Complexity	Individual quality rules only	NOT DONE	SonarQube: c
Technical debt	Findings/age/custom reports	NOT DONE	SonarQube: 3
Quality profiles	Suites/packs + managed rules	PARTIAL	security-exte
History / portfolios	Repository and org views	NOT DONE	Repository PR
Secret scanning	Scanning + push protection	DONE	Enabled; 0 op
Dependency risk	Dependabot + review + graph	DONE	Review passe
AI findings / Autofix	Code Quality AI	NOT DONE	ai_findings_op
Custom security rules	Custom CodeQL packs/models	NOT DONE	security-exte

6. GitHub Code Quality configuration proof

Configuration	Observed value
State	configured
Language	csharp
Schedule	weekly
Runner	standard
AI findings	disabled

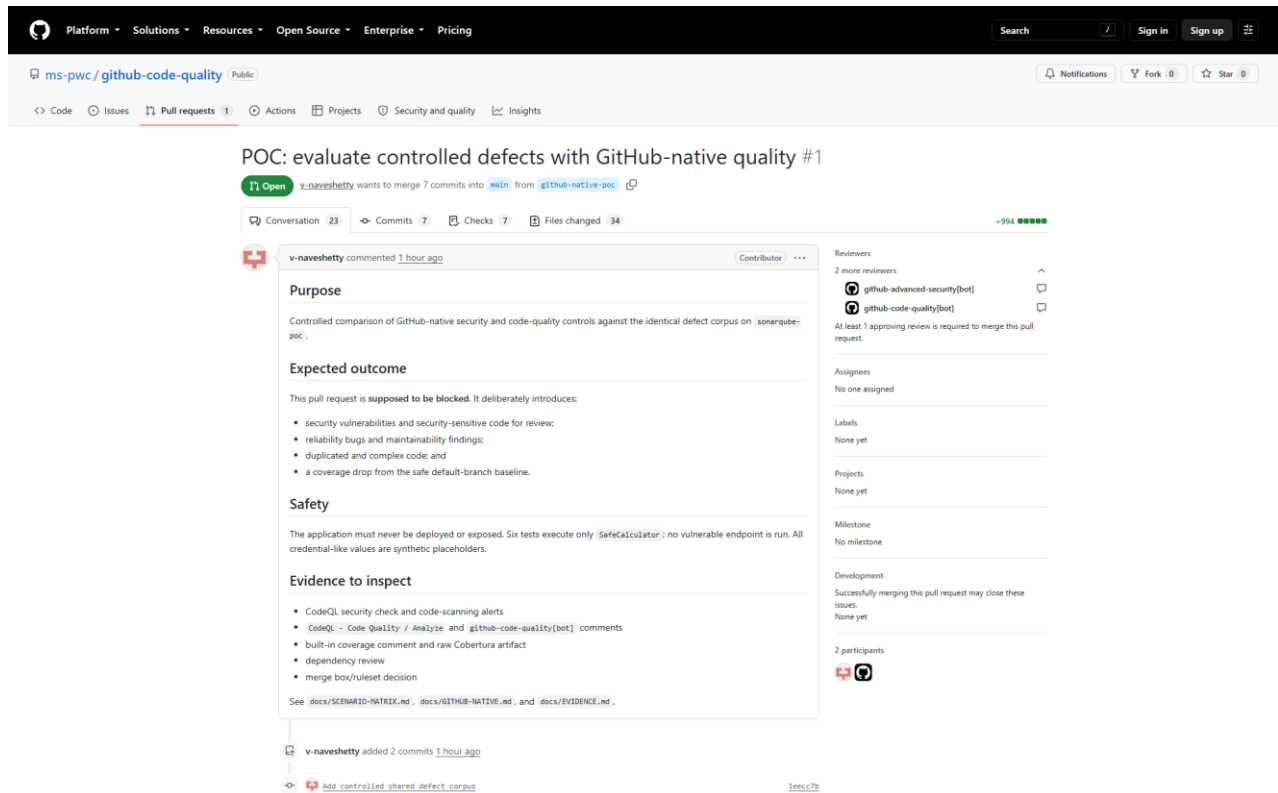


Figure 1. Public POC pull request showing the deliberately vulnerable change set and repository-native quality workflow context.

Live pull request: <https://github.com/ms-pwc/github-code-quality/pull/1>

7. Security implementation and CodeQL proof

Verified security result

11 CodeQL alerts: 2 critical, 4 high, and 5 medium. The CodeQL security check failed and contributed to the blocked merge.

Severity	Finding	CodeQL rule	Location
CRITICAL	Uncontrolled command line	cs/command-line-injection	src/QualityDemo.Api/Controllers/VulnerableController.cs:42
CRITICAL	Untrusted XML is read insecurely	cs/xml/insecure-dtd-handling	src/QualityDemo.Api/Controllers/VulnerableController.cs:86
HIGH	SQL query built from user-controlled sources	cs/sql-injection	src/QualityDemo.Api/Controllers/VulnerableController.cs:31
HIGH	Regular expression injection	cs/regex-injection	src/QualityDemo.Api/Controllers/VulnerableController.cs:95
HIGH	Encryption using ECB	cs/ecb-encryption	src/QualityDemo.Api/Services/SecurityHotspotExamples.cs:36
HIGH	Uncontrolled data used in path expression	cs/path-injection	src/QualityDemo.Api/Controllers/VulnerableController.cs:55
MEDIUM	Missing XML validation	cs/xml/missing-validation	src/QualityDemo.Api/Controllers/VulnerableController.cs:86
MEDIUM	URL redirection from remote source	cs/web/unvalidated-url-redirection	src/QualityDemo.Api/Controllers/VulnerableController.cs:74
MEDIUM	Log entries created from user input	cs/log-forging	src/QualityDemo.Api/Controllers/VulnerableController.cs:101
MEDIUM	Cookie 'Secure' attribute is not set to true	cs/web/cookie-secure-not-set	src/QualityDemo.Api/Controllers/VulnerableController.cs:108

Severity	Finding	CodeQL rule	Location
MEDIUM	Cookie 'HttpOnly' attribute is not set to true	cs/web/cookie-httponly-not-set	src/QualityDemo.Api/Controllers/VulnerableController.cs:108

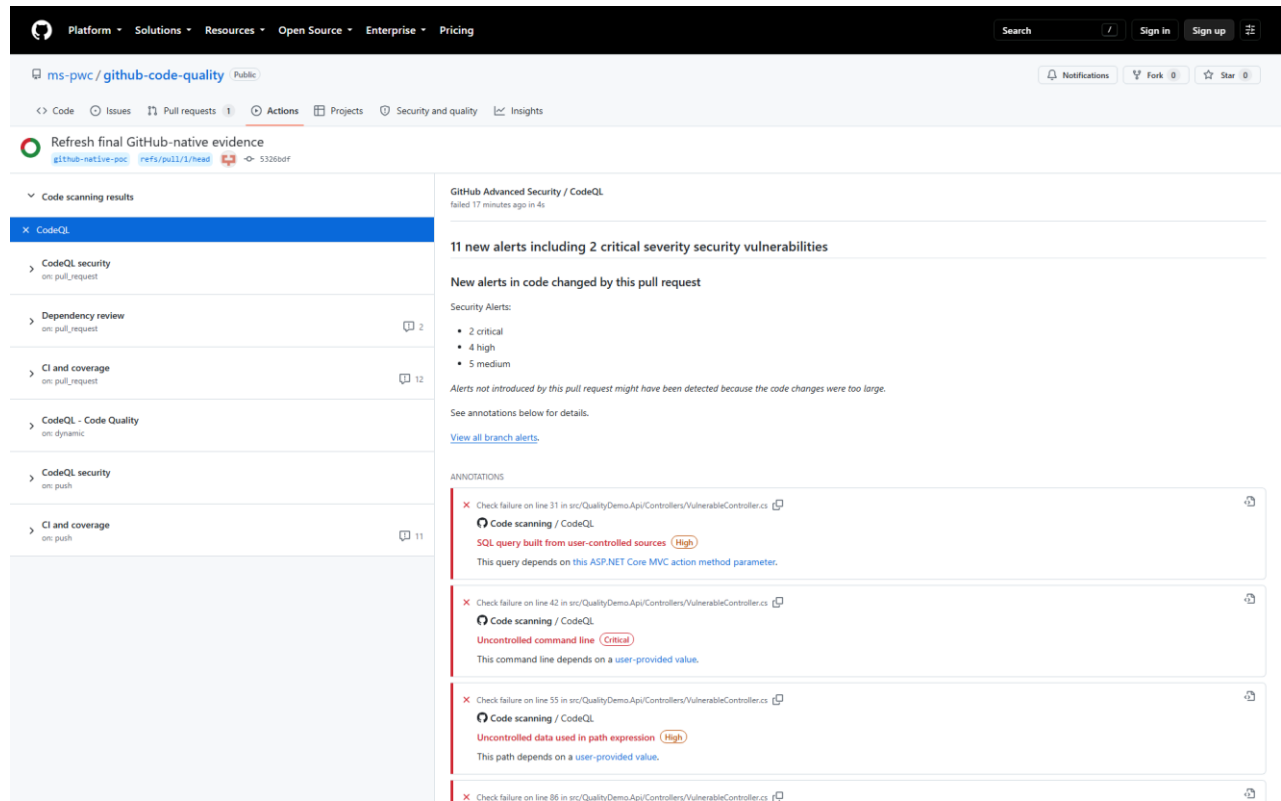


Figure 2. CodeQL check proof: 11 new alerts, including 2 critical vulnerabilities. Severity totals and the branch-alert link are visible.

Branch code-scanning alerts: <https://github.com/ms-pwc/github-code-quality/security/code-scanning?query=pr%3A1+tool%3ACodeQL+is%3Aopen>

Planned security scenarios not observed as CodeQL alerts

NOT DONE / not detected in this corpus

No CodeQL alert was observed for the SSRF fixture, reflected-XSS fixture, response-splitting fixture, MD5 fixture, predictable-random fixture,

8. Reliability and maintainability proof

Verified GitHub Code Quality result

10 inline findings were posted by github-code-quality[bot]: 9 reliability findings and 1 maintainability finding.

Category	Finding	Location
Reliability	Dereferenced variable is always null	src/QualityDemo.Api/Services/ReliabilityExamples.cs:13
Reliability	Invalid string formatting	src/QualityDemo.Api/Services/ReliabilityExamples.cs:49
Reliability	Self-assignment	src/QualityDemo.Api/Services/ReliabilityExamples.cs:54
Reliability	Self-assignment	src/QualityDemo.Api/Services/ReliabilityExamples.cs:55

Category	Finding	Location
Maintainability	Container contents are never accessed	src/QualityDemo.Api/Services/MaintainabilityExamples.cs:41
Reliability	Off-by-one comparison against container length	src/QualityDemo.Api/Services/ReliabilityExamples.cs:25
Reliability	Container contents are never initialized	src/QualityDemo.Api/Services/ReliabilityExamples.cs:67
Reliability	Character passed to StringBuilder constructor	src/QualityDemo.Api/Services/ReliabilityExamples.cs:44
Reliability	Call to ReferenceEquals(...) on value type expressions	src/QualityDemo.Api/Services/ReliabilityExamples.cs:39
Reliability	Potentially dangerous use of non-short-circuit logic	src/QualityDemo.Api/Services/ReliabilityExamples.cs:73

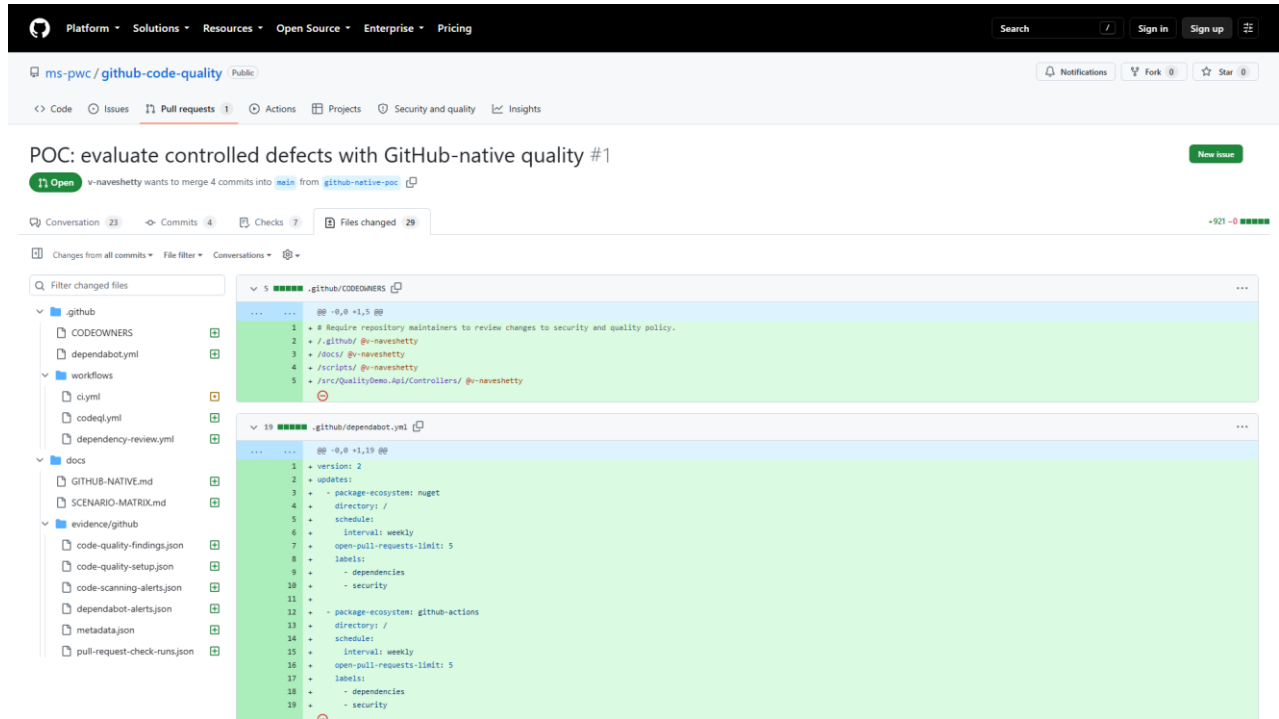


Figure 3. GitHub Code Quality inline finding. The bot identifies a definite null dereference and provides source-level remediation guidance.

First Code Quality inline finding: https://github.com/ms-pwc/github-code-quality/pull/1#discussion_r3801350927

Quality cases not observed

The managed C# rules did not produce an inline finding for every planned smell/reliability case. Examples not observed include possible-null dereference, missing dispose, empty catch, lock(this), explicit GC.Collect, redundant Boolean expressions, nested-if simplification, string concatenation in a loop, aggregate complexity, and duplication. These are recorded as gaps rather than successes.

9. Coverage implementation and gate proof

Coverage evidence	Observed value
Default branch baseline	100%
github-native-poc branch	3%
Coverage drop	97 percentage points
Required workflow threshold	80%
Coverage check conclusion	Failure (intentional)
Managed Restrict code coverage rule	Not configured through public REST API

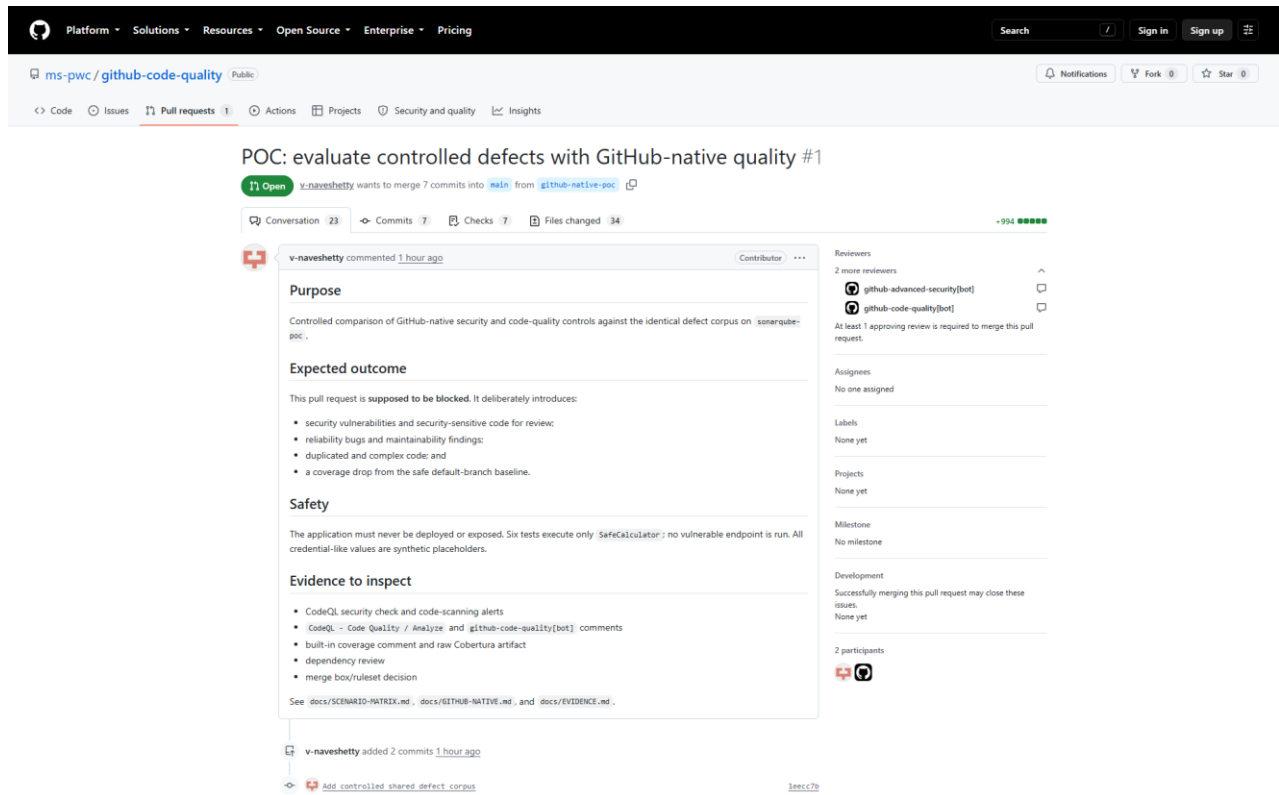


Figure 4. Pull-request overview showing the Code Quality coverage indicator (3% on new code). The built-in bot comment records 3% branch coverage versus the 100% baseline.

Built-in Code Coverage Overview comment: <https://github.com/ms-pwc/github-code-quality/pull/1#issuecomment-5324136729>

Why this is equivalent in outcome

Neither SonarQube nor GitHub runs tests by itself. Coverlet generates the report. SonarQube imports OpenCover; GitHub imports Cobertura.

10. Quality gate and merge-control proof

Verified enforcement outcome

Pull request merge state: BLOCKED. CodeQL security and the 80% coverage check failed. Required checks and review policy remain enforced.

Ruleset control	Configuration
Enforcement	active
Target	Default branch
Pull request	1 approval; code owner; last-push approval; re
Required checks	Build, test, and collect coverage; Analyze C# se
Code scanning threshold	CodeQL high_or_higher; errors
Force push/deletion protection	Enabled
Emergency bypass	Organization administrators; audited
Native Code Quality severity threshold	Not configured

Ruleset control	Configuration
Native coverage threshold rule	Not configured

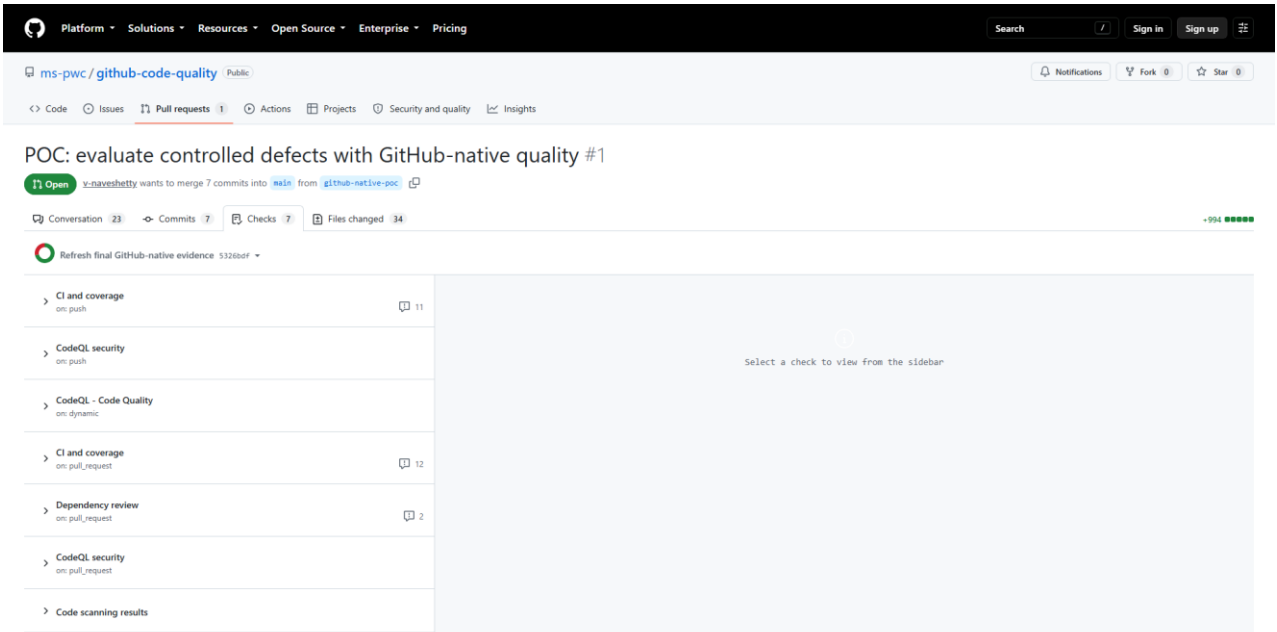


Figure 5. Pull-request checks proof. GitHub-native analysis ran in Actions; strict security and coverage policies prevent a successful merge.

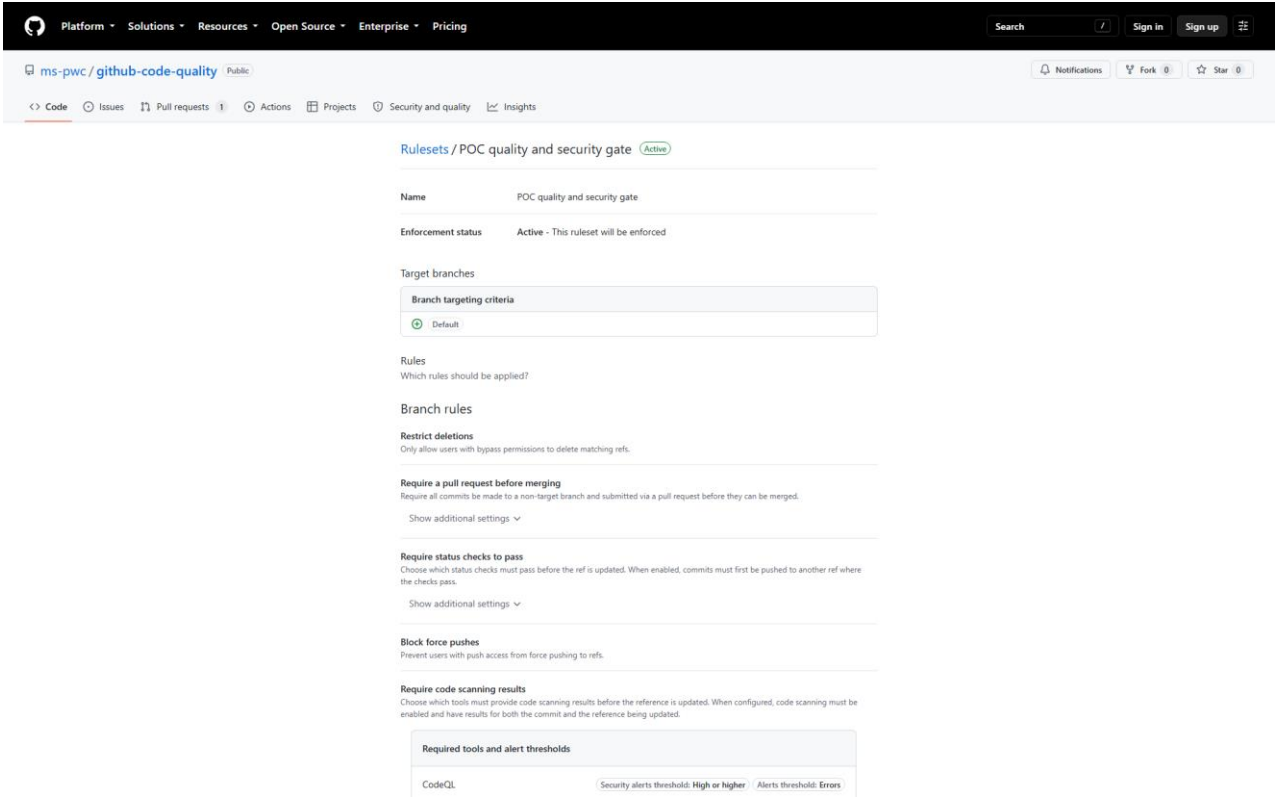


Figure 6. Active repository ruleset page. The rule applies to the default branch and remains auditable in repository settings.

Active ruleset: <https://github.com/ms-pwc/github-code-quality/rules/20972666>

11. Complementary GitHub-native security controls

Control	Observed state	Evidence / result
Secret scanning	enabled	0 open alerts
Push protection	enabled	Blocks supported se
Secret non-provider patterns	disabled	Unavailable/disabled
Secret validity checks	disabled	Unavailable/disabled
Dependabot security updates	enabled	0 open alerts
Dependency review	Required PR check	Passed
Dependency graph / SBOM	Enabled	10 packages invento

The screenshot shows the GitHub Actions interface for the repository 'ms-pwc/github-code-quality'. The 'CodeQL security' workflow is selected in the left sidebar. The main area displays a list of 10 workflow runs. Each run is represented by a green circle with a checkmark, indicating a successful status. The runs include details such as the workflow name, the event that triggered it, the actor, and the duration. The runs are ordered chronologically, with the most recent at the top.

Run #	Workflow Name	Event	Actor	Status	Duration
10	POC: evaluate controlled defects with GitHub-native quality	CodeQL security #10: Pull request #1 synchronize by v-naveshetty	github-native-poc	Success	19 minutes ago
9	Refresh final GitHub-native evidence	CodeQL security #9: Commit 532ba0f pushed by v-naveshetty	github-native-poc	Success	19 minutes ago
8	POC: evaluate controlled defects with GitHub-native quality	CodeQL security #8: Pull request #1 synchronize by v-naveshetty	github-native-poc	Success	25 minutes ago
7	Add GitHub and ruleset screenshots	CodeQL security #7: Commit 52ba0f2 pushed by v-naveshetty	github-native-poc	Success	25 minutes ago
6	POC: evaluate controlled defects with GitHub-native quality	CodeQL security #6: Pull request #1 synchronize by v-naveshetty	github-native-poc	Success	46 minutes ago
5	Enforce pull request coverage threshold	CodeQL security #5: Commit 0ba3e6e pushed by v-naveshetty	github-native-poc	Success	46 minutes ago
4	POC: evaluate controlled defects with GitHub-native quality	CodeQL security #4: Pull request #1 synchronize by v-naveshetty	github-native-poc	Success	52 minutes ago
3	Record verified GitHub-native evidence	CodeQL security #3: Commit a2723ff pushed by v-naveshetty	github-native-poc	Success	52 minutes ago
2	POC: evaluate controlled defects with GitHub-native quality	CodeQL security #2: Pull request #1 opened by v-naveshetty	github-native-poc	Success	Today at 11:15 AM
1	Configure GitHub-native quality and security analysis	CodeQL security #1: Commit 03a1a6d pushed by v-naveshetty	github-native-poc	Success	Today at 11:14 AM

Figure 7. CodeQL workflow history in GitHub Actions. Security analysis runs are repository-native and auditable.

12. SonarQube benchmark result on the same source

SonarQube was run locally on the byte-identical sonarqube-poc branch to provide a benchmark—not to claim identical rules or counts.

SonarQube metric
Quality Gate
Vulnerabilities
Bugs
Code Smells
Total issues
Coverage

SonarQube metric

Duplication

Complexity

Technical debt

Security Hotspots

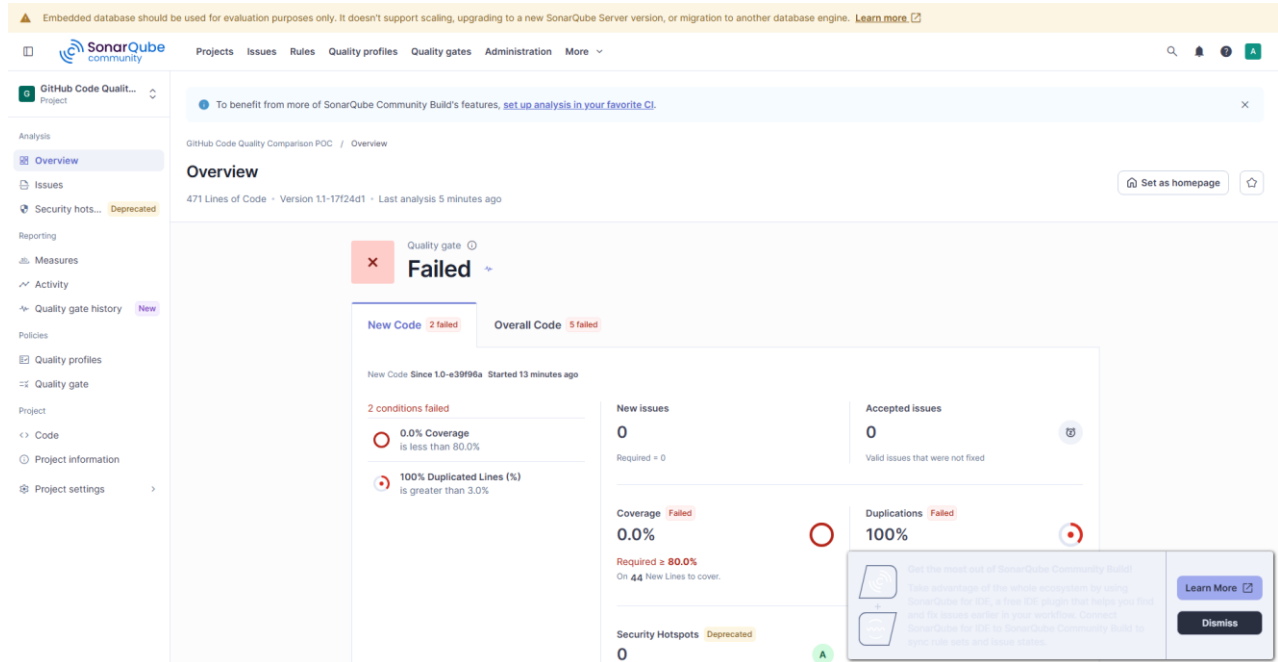


Figure 8. SonarQube dashboard on the same source: failed Quality Gate, 10 vulnerabilities, 8 bugs, 52 code smells, 2.7% coverage, and 21.6% duplication.

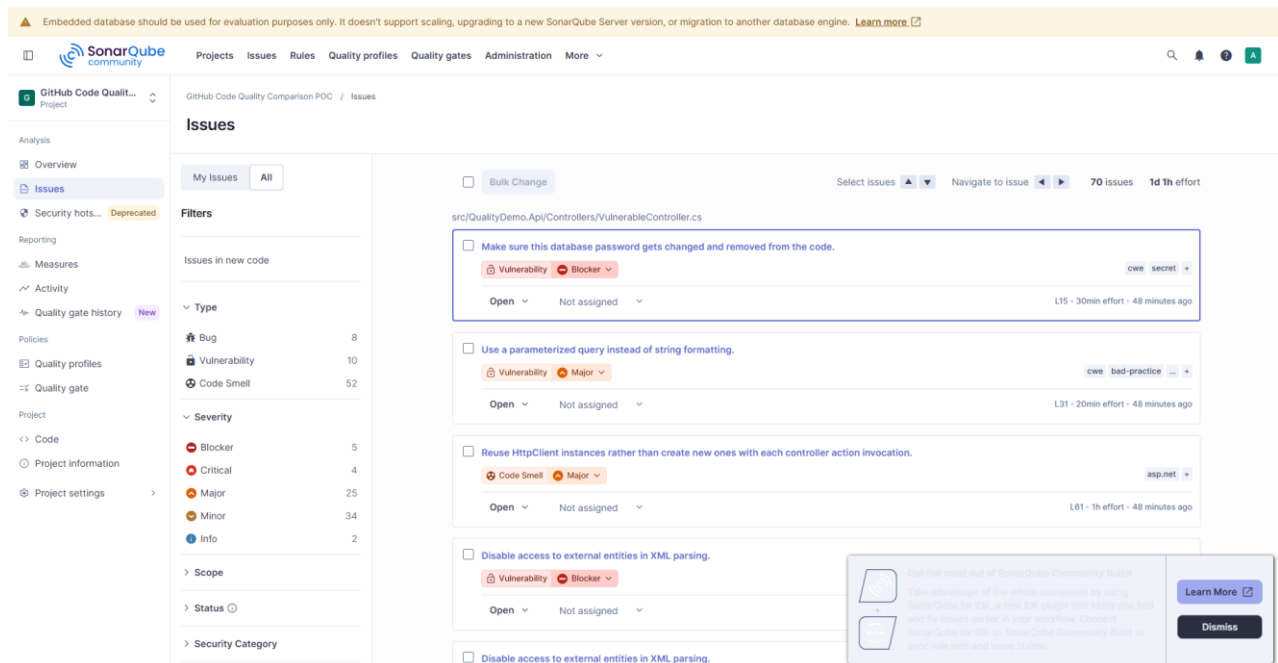


Figure 9. SonarQube Issues view showing the broader 70-issue result and classic Bug/Vulnerability/Code Smell taxonomy.

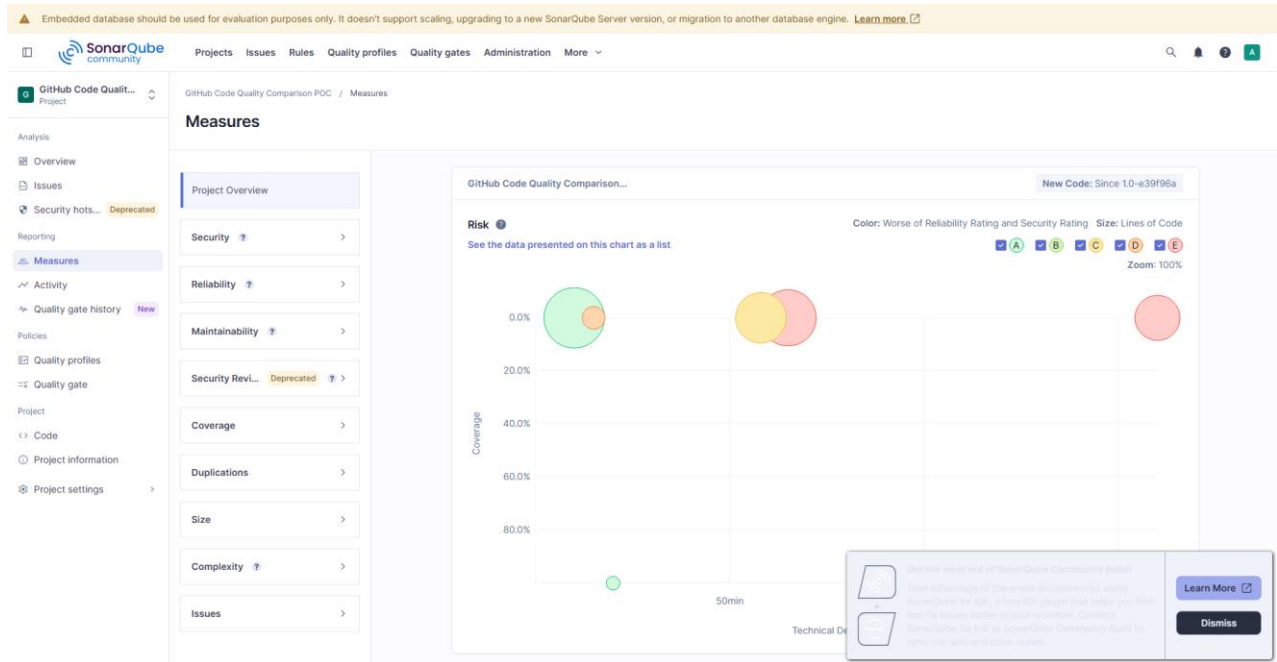


Figure 10. SonarQube Measures view proving coverage, duplication, size, complexity, and technical-debt metrics.

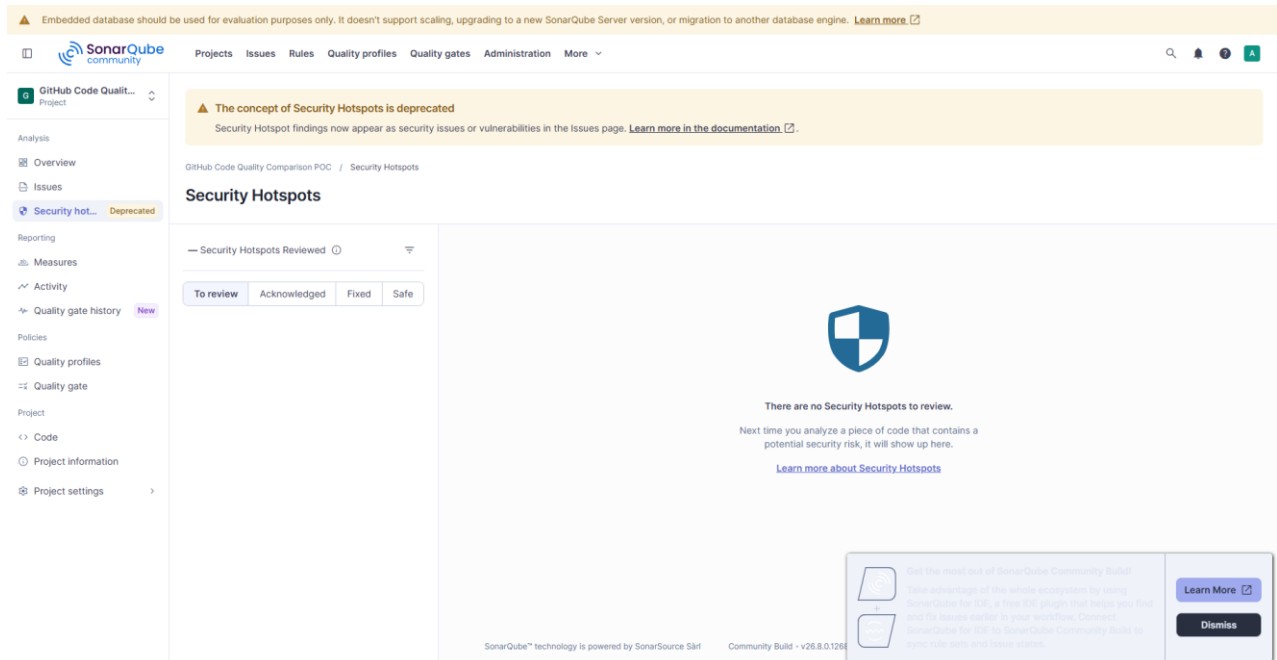


Figure 11. SonarQube Security Hotspots page showing no hotspot emitted by the installed 26.8 C# rule catalog. This limitation is recorded, not hidden.

13. Explicit NOT DONE and PARTIAL register

The following items must not be described as implemented

They are absent, structurally different, disabled, or not empirically validated in this repository.

Item	Status
Exact Security Hotspot Reviewed/Safe lifecycle and metric	PARTIAL
Native duplication percentage and gate	NOT DONE

Item	Status
Aggregate cyclomatic/cognitive complexity and gate	NOT DONE
Remediation-time technical-debt model	NOT DONE
Central inherited Quality Profile	PARTIAL
Custom CodeQL query/model packs	NOT DONE
Native Require code quality results threshold	NOT DONE
Native Restrict code coverage ruleset	NOT DONE
AI findings / Copilot Autofix	NOT DONE
Organization/portfolio trends and audit exports	NOT DONE
Secret non-provider patterns / validity checks	NOT DONE
Every planned CodeQL security fixture	PARTIAL
Every Sonar reliability / maintainability rule	PARTIAL
Commercial/licensing and policy approval	NOT DONE

14. SonarQube retirement decision framework

Approve removal when all conditions below are true

The repository uses supported languages/frameworks; mandatory rules are mapped; coverage and merge controls are active; gaps are accepted

Do not remove SonarQube when any condition below is true

- A mandatory language or framework is unsupported by CodeQL or GitHub Code Quality.
- A mandatory custom Sonar rule/profile has no GitHub or custom-check equivalent.
- Duplication, aggregate complexity, remediation debt, hotspot-reviewed percentage, or portfolio history is required and no compensating control is approved.
- GitHub Code Quality entitlement, Actions capacity, or AI-credit governance is not approved.
- Required organization dashboards, audit exports, retention, or separation of duties are not verified.
- The ruleset can be bypassed without an accepted organization-wide exception process.

Recommended production rollout

1. Export every SonarQube Quality Gate condition, active Quality Profile rule, exclusion, plugin, and report dependency.
2. Classify each as CodeQL, Code Quality, coverage, ruleset/check, custom query/check, accepted retirement, or blocking gap.
3. Pilot on a representative repository and compare scenario detection, not raw counts.
4. Run rulesets in evaluate mode where available; activate after check contexts are stable.
5. Obtain security, engineering, audit, licensing, and policy approval for every accepted gap.
6. Retire SonarQube repository by repository, retain evidence, and maintain a rollback path.

15. Demonstration script

1. Open the repository and explain the safe main baseline plus the byte-identical POC branches.

2. Open PR #1 and show security, reliability, maintainability, duplication, complexity, and untested code fixtures.
3. Open the failing CodeQL check and show 11 alerts: 2 critical, 4 high, 5 medium.
4. Open Files changed and show github-code-quality[bot] inline findings.
5. Open the Code Coverage Overview and show 3% versus 100%.
6. Open checks and show the 80% coverage failure and dependency-review success.
7. Open the active ruleset and show PR review, required checks, and high-or-higher CodeQL protection.
8. Show mergeStateStatus BLOCKED.
9. Open SonarQube and show 70 issues, 2.7% coverage, 21.6% duplication, complexity, debt, and failed gate.
10. Finish with the NOT DONE table and explain the conditional retirement decision.

Closing statement

The POC proves that the main pre-merge developer workflow can move to GitHub-native controls for this supported C# repository. It also proves that there remain explicit gaps.

16. Evidence and navigation index

Evidence	Where to look
POC pull request	https://github.com/ms-pwc/github-code-quality/pull/1
CodeQL branch alerts	https://github.com/ms-pwc/github-code-quality/security/code-scanning?query=pr%3A1+tool%3ACodeQL+is%3Aopen
GitHub Code Quality repository view	https://github.com/ms-pwc/github-code-quality/security/code-quality
Coverage bot comment	https://github.com/ms-pwc/github-code-quality/pull/1#issuecomment-5324136729
Active ruleset	https://github.com/ms-pwc/github-code-quality/rules/20972666
Actions history	https://github.com/ms-pwc/github-code-quality/actions
GitHub-native branch	https://github.com/ms-pwc/github-code-quality/tree/github-native-poc
SonarQube branch/evidence	https://github.com/ms-pwc/github-code-quality/tree/sonarqube-poc
Local SonarQube dashboard	http://localhost:9000/dashboard?id=github-code-quality-poc

Repository: <https://github.com/ms-pwc/github-code-quality>

POC pull request: <https://github.com/ms-pwc/github-code-quality/pull/1>

Ruleset: <https://github.com/ms-pwc/github-code-quality/rules/20972666>

Code scanning: <https://github.com/ms-pwc/github-code-quality/security/code-scanning?query=pr%3A1+tool%3ACodeQL+is%3Aopen>

17. Official product references

Reference	URL
GitHub CodeQL	https://docs.github.com/en/code-security/code-scanning/introduction-to-code-scanning/about-code-scanning
GitHub Code Quality	https://docs.github.com/en/code-security/concepts/code-quality/code-quality
CodeQL-powered Code Quality	https://docs.github.com/en/code-security/reference/code-quality/codeql-detection
Built-in code coverage	https://docs.github.com/en/code-security/how-to/maintain-quality-code/set-up-code-coverage
Repository rulesets	https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets

Reference	URL
Secret Scanning	https://docs.github.com/en/code-security/secret-scanning/introduction/about-secret-scanning
Dependency review	https://docs.github.com/en/code-security/supply-chain-security/understanding-your-software-supply-chain/al
SonarQube Server	https://docs.sonarsource.com/sonarqube-server/latest/
SonarQube quality gates	https://docs.sonarsource.com/sonarqube-server/quality-standards-administration/managing-quality-gates
SonarQube quality profiles	https://docs.sonarsource.com/sonarqube-server/instance-administration/analysis-functions/quality-profiles

End of report

Generated from live GitHub API evidence and committed SonarQube API/screenshot evidence on 18 August 2026.